

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work n. 1
of Operating Systems

The OS as a Resource Manager

Supervisers:

Dumitrașcu Marius, *University assistant*

Reițman Patricia, *University assistant*

DISA, FCIM, UTM

Chișinău – 2026

Table of Contents

1 Introduction **3**

2 Before we start **3**

3 Exploting the OS **3**

3.1 Files and directories 3

3.2 Processes 5

3.3 Memory 7

3.4 Devices and storage 8

4 CONCLUSIONS **9**

1 Introduction

In this laboratory work, we will explore the operating system as a manager that stands between programs and hardware. We will see the four core jobs of any OS (managing files, processes, memory and devices) happen live in the terminal, and we will document our observations.

2 Before we start

Before we start, we need to log in the Ubuntu in the Virtual Machine. Then, create a working folder and keep a file called report.md opened beside the terminal.

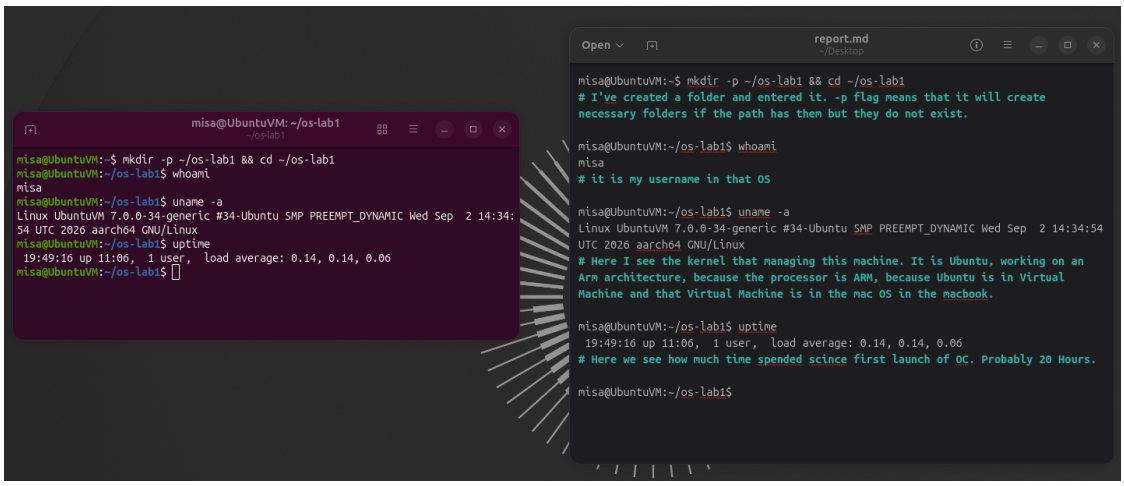


Figure 2.1 - Basic configuration of OS and observations in report.md

So, my username is 'misa', I use Ubuntu in Virtual MAchine which is running on macbook real machine. Ubuntu is running already almost 20 hours.

3 Exploting the OS

3.1 Files and directories

The OS turns a raw disk into a tidy tree of named files with owners and permissions. We will explore that tree and change it.

To see where we are now, we use `pwd` command. The `ls -la` will show the root of the whole tree. `-l` flag means to show in a list, the `-a` flag meand to show all, even hidden file and folders.

`ls -la ~` will show the tree of the home directory as `~` is the alias for that directory. The `cd /etc` will enter into the etc folter and `ls | head` will show first 10 results of ls, as the result of `ls` command goes to `head` command and this command shows only top 10 results.

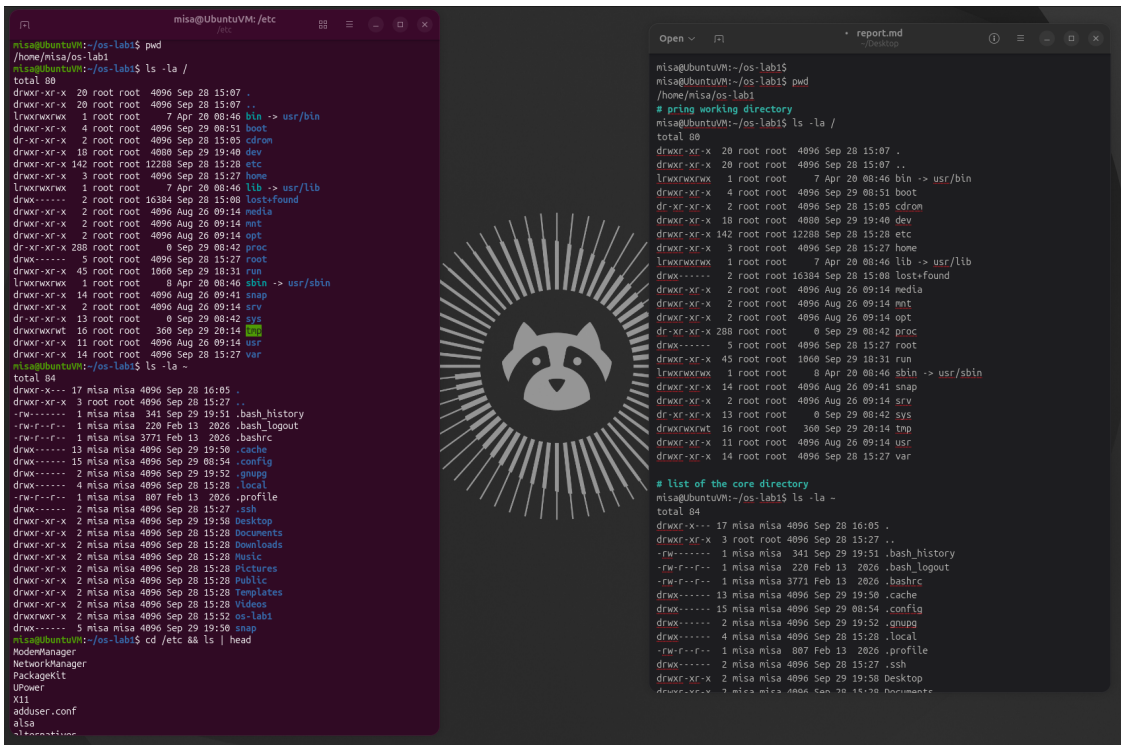


Figure 3.1 - Exploring the file tree

Moving forward, now we will not just explore, but create, copy, rename and remove something and see the tree changes. `cd os-lab1` will enter the `os-lab1` folder. Then we will create a `demo` folder. There create a file named `note.txt` containing the text `"hello operating systems"`. Then we will create a copy of that file, rename the copy, see the list of tree in that directory and finally remove the copied renamed file.

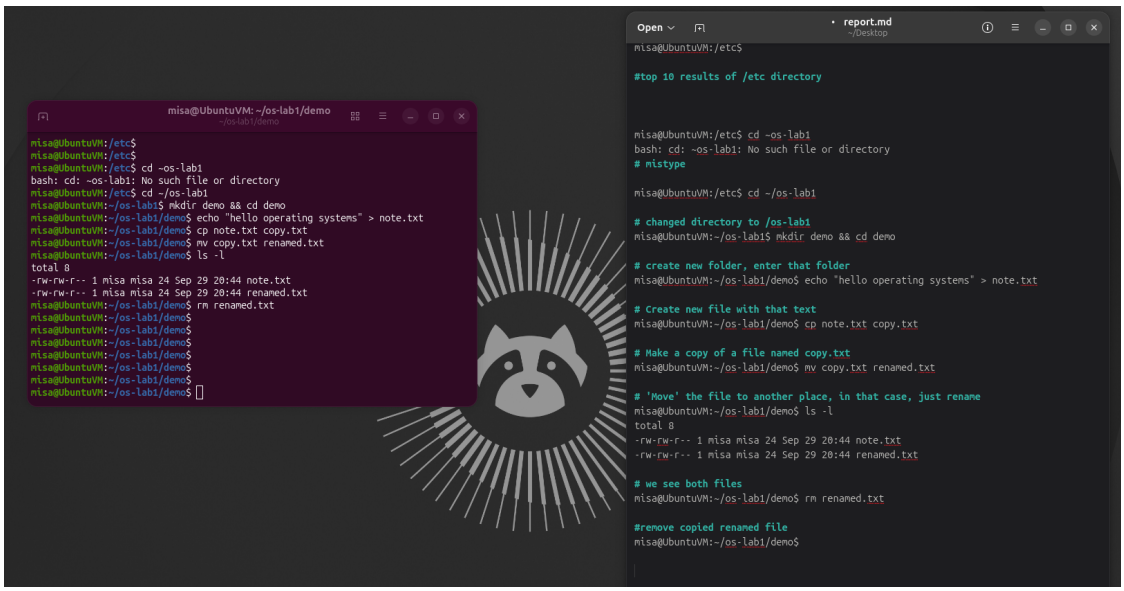


Figure 3.2 - Using the file tree

Now we explore the ownership permissions. The OS decides who may touch what. We will `change`

`mod` of file and see what happens in the ouptut of `ls -l` command.

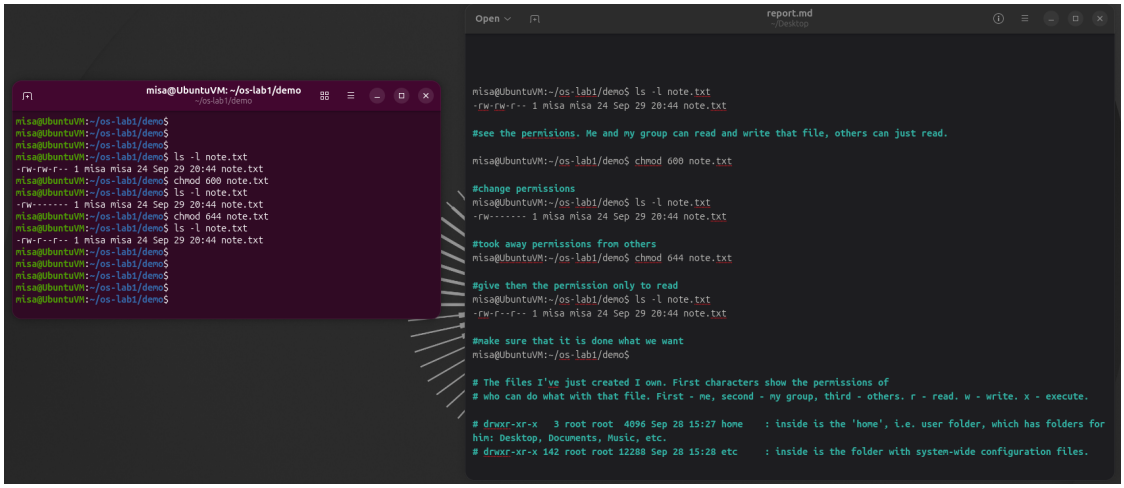


Figure 3.3 - Edit and explore permissions

3.2 Processes

Every running program is a process that OS created, isolated and schedules. We will find them, start one and stop it.

Firstly, we will see the processes that OS is currently juggling.

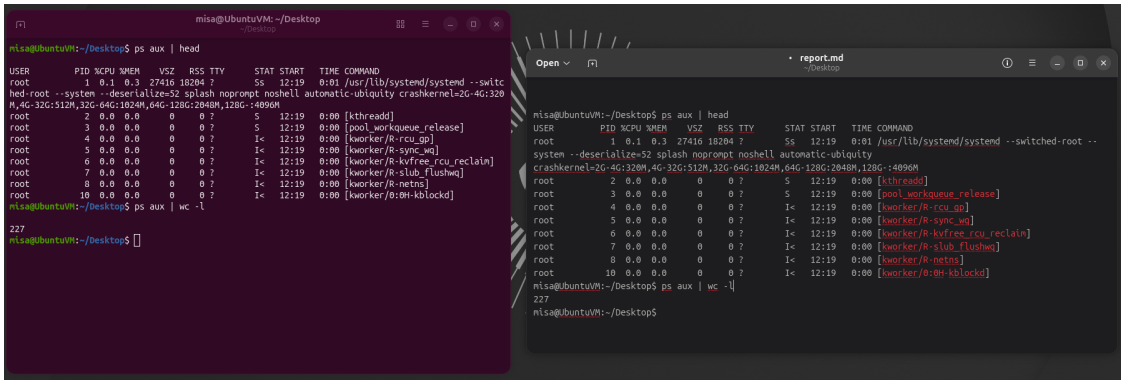


Figure 3.4 - List the processes the OS is currently juggling

The OS is currently juggling 227 processes.

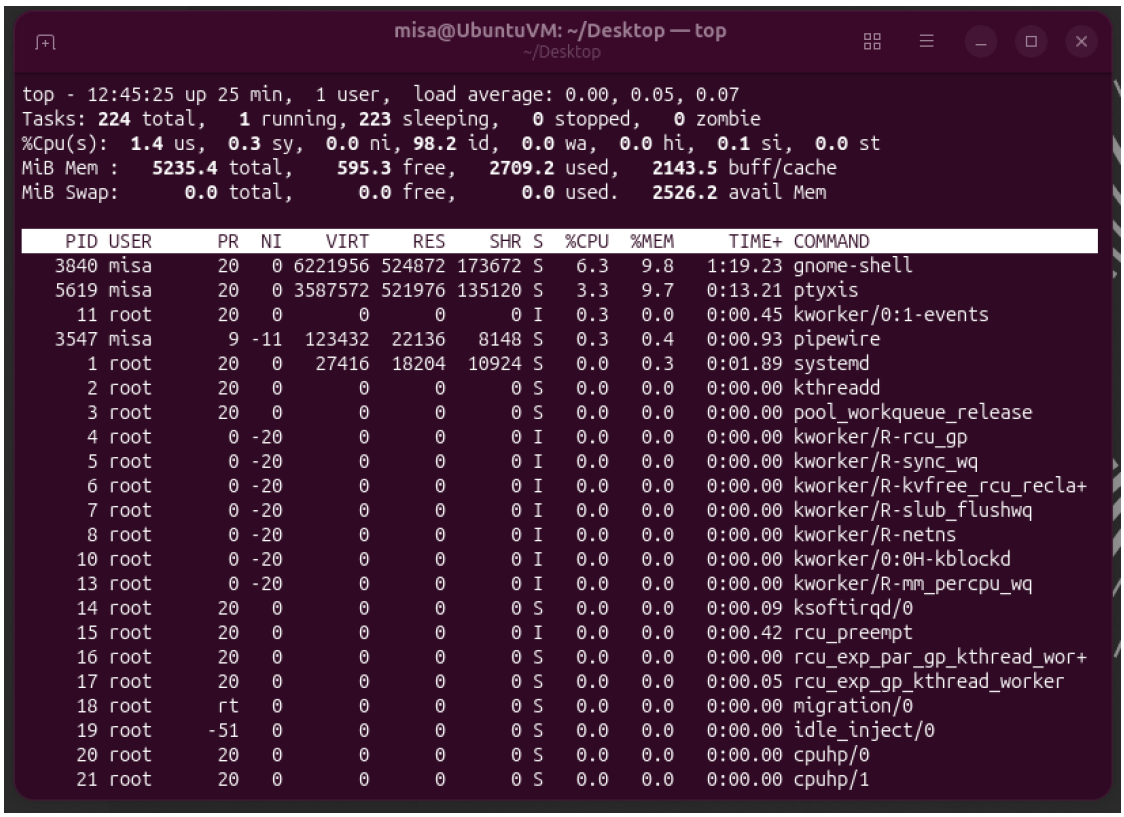


Figure 3.5 - Output of `top` command

Now start the background process, find it and stop it.

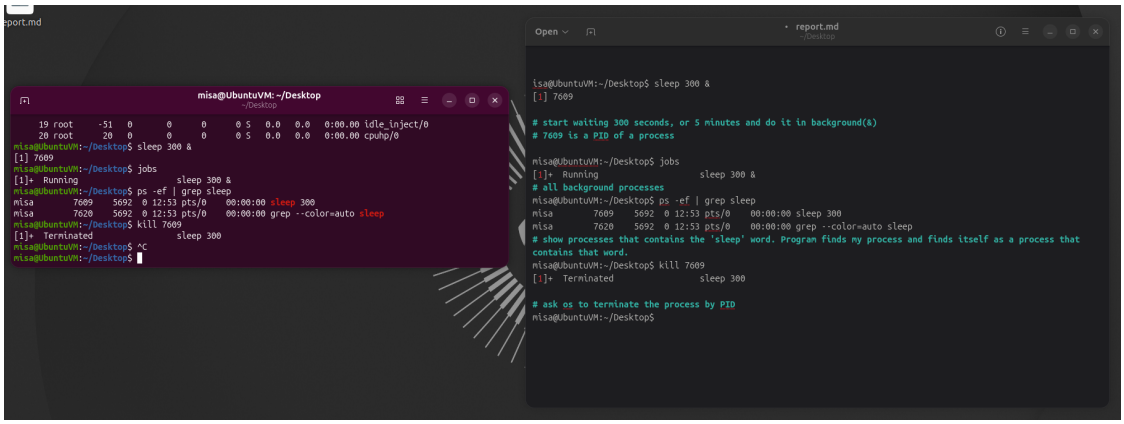


Figure 3.6 - Start background process, find it and stop it

The `ps -ef` shows all processes in a full format, the `grep sleep` leaves only processes with the name `sleep`.

Now peek at what the kernel exposes about a process through the `/proc pseudo-filesystem`.

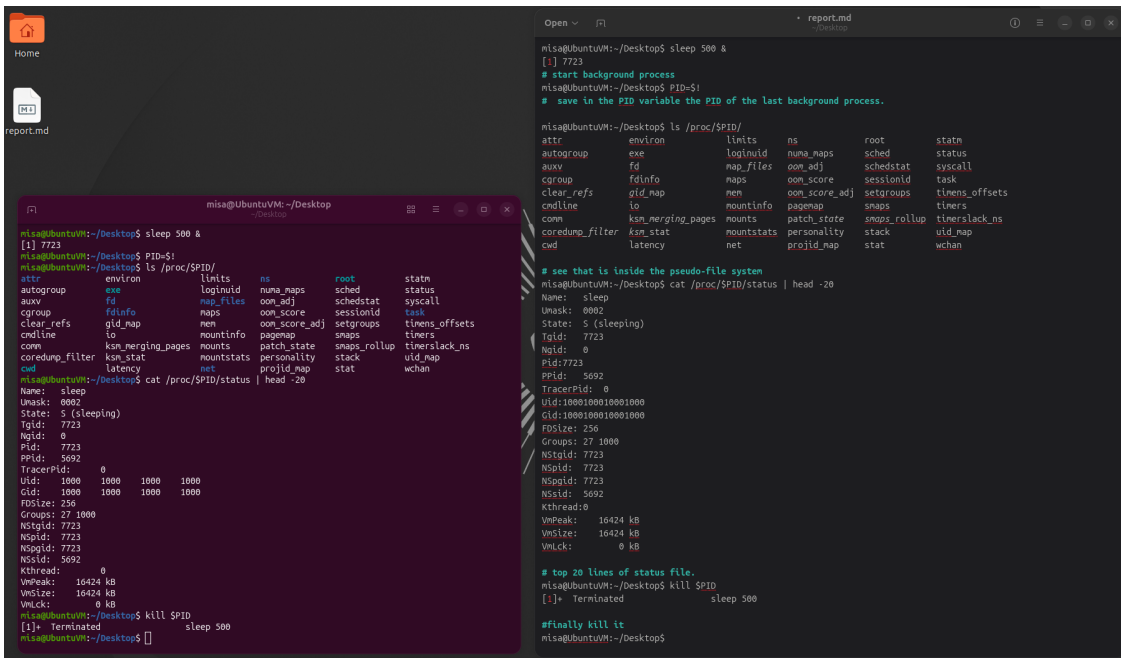


Figure 3.7 - peek at what the kernel exposes about a process through the `/proc pseudo-filesystem`

`/proc` is a pseudo-filesystem that provides information about running processes and the current state of the Linux kernel through a file-like interface.

We notice that the process with PID 1 is `systemd`, the first userspace process started by the Linux kernel that manages system services. The idle VM had roughly 220–230 running processes. We also notice that the `sleep` process had the state `S (sleeping)`, meaning that it was waiting and not actively using the CPU.

3.3 Memory

The OS hands each process its own view of memory and shares the physical RAM between them.

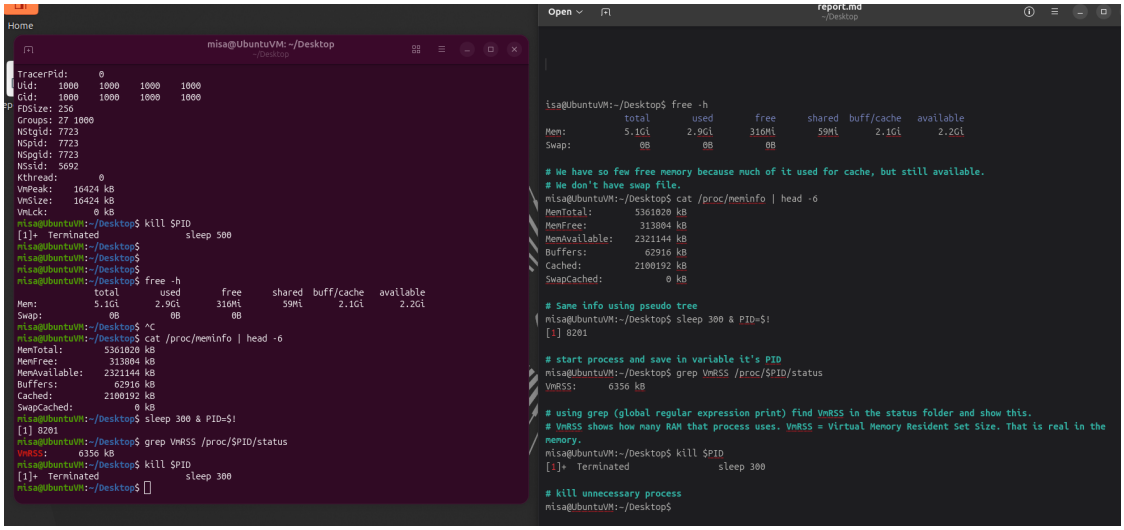


Figure 3.8 - Memory management

We notice that the VM has 5.1 GiB of total RAM, with only about 316 MiB completely free. However, about 2.2 GiB is still available because a significant part of the memory is used for cache and can be reclaimed when needed. We also notice that no swap is configured, meaning that no disk space is currently used as an extension of RAM. Using the `/proc` pseudo-filesystem, we find that a simple `sleep` process has a VmRSS (Virtual Memory Resident Set Size) of 6356 kB, or about 6.2 MiB of physical RAM. This is surprisingly large considering that the `sleep` process does almost no actual work.

3.4 Devices and storage

The OS hides every disk, terminal and device behind uniform file-like interfaces. That is the ”everything is a file” idea of UNIX.

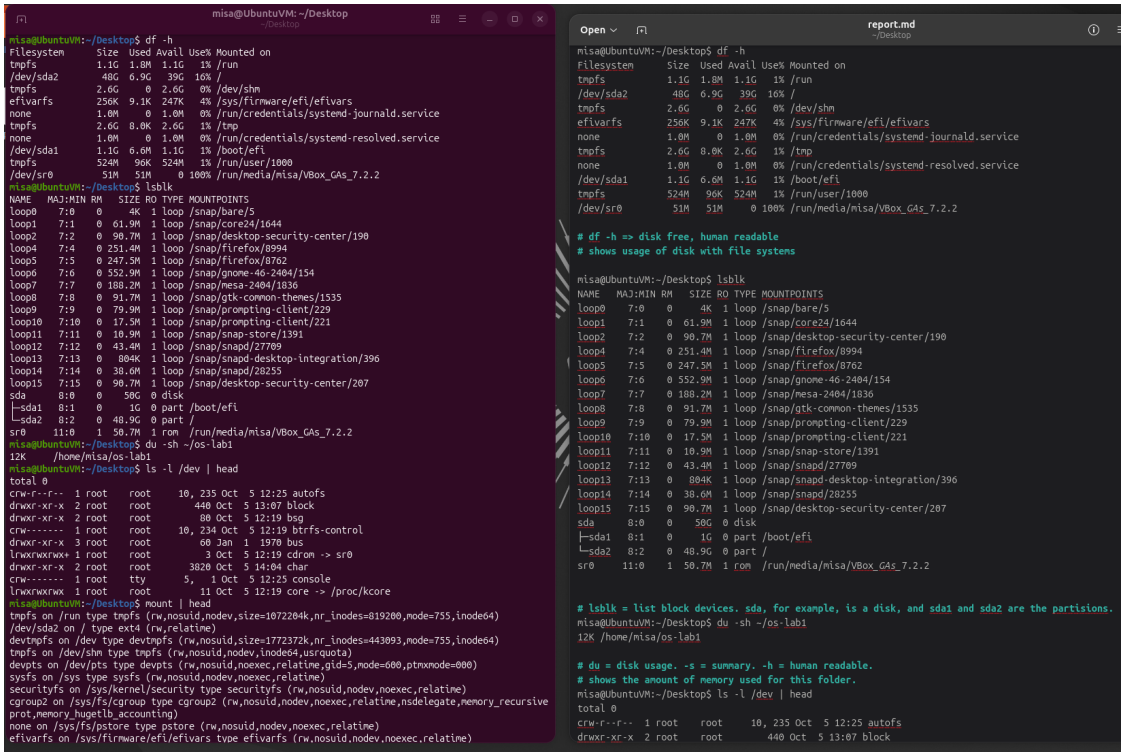


Figure 3.9 - Devices and storage

We notice that the main disk `sda` has two partitions, with `/dev/sda2` containing the root filesystem `/` and `/dev/sda1` mounted at `/boot/efi`. The root filesystem has about 48 GiB of space, of which approximately 6.9 GiB is used. We can also notice that the `os-lab1` directory occupies only 12 KiB of disk space. The `/dev` directory provides file interfaces to devices; for example, `/dev/cdrom` represents the virtual CD/DVD drive, while `/dev/console` represents the system console and provides an interface for its input and output. The mounted filesystems show that different filesystems, such as `ext4`, `tmpfs`, `devtmpfs`, and `sysfs`, are attached to different points of the same directory tree. From this, we can notice the Unix concept that “everything is a file”, meaning that Linux provides access to many system resources, including hardware devices, through a common file-like interface.

4 CONCLUSIONS

During this laboratory work, we observed how Linux manages four main system resources: files, processes, memory, and I/O devices. File management was explored using `ls -la`, which allowed us to inspect the file system, ownership, and permissions; process management was observed using `ps aux`, which showed the processes currently managed by the OS. Memory management was examined using `free -h`, which showed how physical RAM is distributed between used, free, and cached memory, while I/O devices and storage were explored using `df -h`, which showed the mounted filesystems and their disk usage. Overall, the laboratory work demonstrated how the operating system acts as a resource manager and provides common interfaces through which programs and users can interact with system resources.